01 数字电路与 RTL

本册目标：读懂电路，再写代码

电脑最终依靠电路处理信息。对入门学习而言，可以把导线上的低电平记为 0、高电平记为 1；这里的 0 和 1 表示逻辑状态，并非所有芯片都使用同一个实际电压。

电路里有两类基本构件：一类根据当前输入计算输出，另一类能够记住之前的值。前者组成组合电路，后者让系统拥有状态。

先建立三个概念

概念	通俗解释	本课题中的例子
信号	导线上随时间变化的值	“请求启动”“已授权”“要求停止”
逻辑	输入如何决定输出	只有请求且授权时才放行
状态	电路保存的历史信息	已收到停止命令，之后继续保持停止

RTL 是 Register Transfer Level，即寄存器传输级。它描述寄存器保存什么、在什么时刻更新，以及寄存器之间怎样计算数据。RTL 是一种描述层次，Verilog / SystemVerilog 是常用的描述语言。

软件与 RTL 的思考方式

软件通常让处理器按照指令执行任务；RTL 用代码描述要构成怎样的电路。两个独立的硬件模块可以持续同时工作，代码里的书写顺序不等于硬件逐条运行顺序。

例如两个连续赋值会描述两条并行逻辑路径：

```verilog
assign p = a & b;
assign q = c | d;
```

assign 表示持续连接逻辑；& 是按位与，| 是按位或，分号表示语句结束。只要相关输入改变，仿真器就会按规则更新输出；真实硬件还存在传播延迟。

> 必学：先把代码画成“输入 → 逻辑 → 输出”。本册所有电路都很小，不要求理解完整 CPU。

新增的“语法补充 V1-V7”在本册后半部分，带独立书签。第一次读代码时先查 V1-V3，遇到模块连接和仿真文件时查 V4-V5，V6-V7 用于动手巩固。示例中的片段不一定能单独编译，完整可运行文件另行标出。

1.1 二进制、位宽与数值

一个 bit 能表达两个状态

bit（位）就是一个 0 或 1。4 个 bit 组成的二进制数 1010，从右到左的位权是 1、2、4、8，所以十进制值为 8 + 0 + 2 + 0 = 10。8 个 bit 通常称为一个字节。

二进制	十进制	解释
0000	0	所有位都是 0
0011	3	2 + 1
1000	8	最高位为 1
1111	15	8 + 4 + 2 + 1

n 位无符号整数的范围是 0 到 2 的 n 次方减 1。“无符号”表示这里只解释非负数；有符号数还需要规定正负数的编码，本周选读即可。

代码里的写法

```VERILOG
wire [3:0] value;
assign value = 4'b1010;
```

wire 表示用于连接的信号；[3:0] 表示有编号 3、2、1、0 共四位；4'b1010 表示“4 位，二进制，数值 1010”。同一个值也能写成 4'd10，其中 d 表示十进制。

为什么必须关心位宽

4 位只能保存 0 到 15。如果把十进制 16 的二进制 10000 存入 4 位，只留下低四位 0000。因此本册的 4 位计数器从 15 再加 1 会回到 0。

这叫截断导致的模 16 行为。若做“最多允许创建 15 个实例”的安全计数器，就不能无条件让它回绕，否则计数可能重新变小。可以考虑饱和计数：达到上限后保持上限，并拒绝新的请求。

试一试

1. 0101 是多少？5 位无符号数最多是多少？
2. 4 位的 1111 + 0001，存回 4 位后是什么？
3. 如果实例计数只能增加，你会选回绕还是饱和？为什么？

1.2 逻辑门与真值表

把一句规则翻成布尔表达式

布尔量只有真和假两种值，常用 1 和 0 表示。与表示“同时满足”，或表示“至少一个满足”，非表示“取反”。

A	B	A 与 B	A 或 B	非 A
0	0	0	0	1
0	1	0	1	1
1	0	0	1	0
1	1	1	1	0

上表列出了所有输入组合及其输出，叫真值表。验证简单组合电路时，可以把所有组合遍历一遍。

例：请求且授权才放行

```verilog
assign allow = request && authorized;
```

request 是是否有请求，authorized 是请求是否获得授权。&& 是逻辑与，常用于判断条件。对单 bit 信号，它在本例中与按位与 & 的结果一致；对多位数据，二者含义不同，不要混用。

如果还要求“没有停止状态”，可写为：

```verilog
assign allow = request && authorized && !stopped;
```

! 是逻辑非。stopped=1 时，!stopped=0，所以无论另外两个条件是什么，allow 都为 0。

读代码的方法

先找输出 allow，再从右向左问三个条件：是否请求、是否授权、是否仍可运行。这样可以把表达式重新翻回自然语言。

试一试

1. request=1、authorized=0、stopped=0，allow 是多少？
2. request=1、authorized=1、stopped=1，allow 是多少？
3. “授权或请求就放行”为何不符合原规则？给一个反例。

1.3 组合电路：二选一选择器

输入决定现在的输出

选择器类似铁路道岔：sel=0 时把 a 接到 y，sel=1 时把 b 接到 y。它不记住上一次选择，不需要时钟。

```VERILOG
module mux2(input wire a, b, sel, output wire y);
  assign y = sel ? b : a;
endmodule
```

module 到 endmodule 定义一个电路模块。input 是输入端口，output 是输出端口。模块名 mux2 可理解为“二选一选择器”。三目运算符 条件 ? 真时取值 : 假时取值 直接描述选择关系。

a	b	sel	y
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0

另一种写法，理解即可

```VERILOG
always @(*) begin
  if (sel) y = b;
  else     y = a;
end
```

在传统 Verilog 中，这样赋值的 y 要声明成 reg；这里的 reg 是语言中的变量类型，并不能仅凭这个词断定产生了寄存器。SystemVerilog 常用 logic 与 always_comb 表达此类逻辑。

组合逻辑需要覆盖所有分支。如果只写 if (sel) y = b;，sel=0 时没有说明 y 应是多少，可能推断出“保存旧值”的锁存器。锁存器是另一类存储元件，本周先通过完整赋值避免无意引入。

与课题的联系

“允许访问 / 返回拒绝”也可以先抽象成选择逻辑，但真实访问控制还需要身份、权限来源和不可绕过的执行位置。一个选择器只展示规则如何变成电路。

练习：写出 sel=0 时选 b、sel=1 时选 a 的一行表达式。配套 mux2_tb.sv 会检查全部 8 种输入组合。

1.4 时钟、寄存器与复位

电路怎样记住过去

寄存器可以保存若干位的值。把时钟想成一个反复跳动的节拍信号。本册使用上升沿触发的寄存器：时钟从 0 变 1 的那个瞬间，寄存器根据输入更新；两次上升沿之间通常保持值。

```VERILOG
always @(posedge clk) begin
  if (reset) q <= 1'b0;
  else       q <= d;
end
```

posedge clk 表示时钟上升沿；q 是已保存的状态，d 是准备写入的新值；<= 是非阻塞赋值。这里 reset=1 时，在上升沿把 q 写成 0。

上升沿编号	reset	d	沿后 q
1	1	1	0
2	0	1	1
3	0	0	0
4	0	1	1

复位用于建立已知初始状态。本例是同步复位：需要等到上升沿才生效。异步复位能够在规定的复位信号变化时触发，不等待时钟；它的释放和时序也有工程要求。[C3]

为什么时序代码常用 <=

想象 a、b 是两个寄存器。在同一个时钟沿执行 a <= b; b <= a; 时，两者分别读取沿到来前的旧值，之后一起更新，因而可以交换值。<= 不表示“小于等于比较”，要结合语境区分。

选读：真实电路的时间要求

输入不能在采样瞬间随意抖动；寄存器有建立时间和保持时间。跨时钟域的信号可能需要同步器。仿真里理想的 0/1 跳变，不能直接保证物理电路在任何时刻都可靠。

练习：若 reset 在两个时钟沿之间短暂从 0 变 1 又回到 0，这个同步复位寄存器一定复位吗？

1.5 逐行读懂一个 4 位计数器

本册第一个有状态的完整模块

```VERILOG
module counter4(
  input wire clk, reset, enable,
  output reg [3:0] count
);
  always @(posedge clk) begin
    if (reset) count <= 4'd0;
    else if (enable) count <= count + 4'd1;
  end
endmodule
```

第一到四行定义接口：时钟、复位、计数使能 enable 是输入，4 位 count 是输出。第五行表示只有时钟上升沿才更新状态。

第六行优先处理复位。第七行只有在 enable=1 时加 1。当 reset=0 且 enable=0，代码不安排更新，寄存器保持旧值。这种“未赋值就保持”的写法在时钟触发的寄存器里符合预期；与组合逻辑缺少分支不同。

先预测，再运行

第几个有效上升沿	reset	enable	沿后 count
1	1	1	0
2	0	1	1
3	0	0	1
4	0	1	2

复位与 enable 同时为 1 时，先命中的 reset 分支决定结果，所以得到 0，不是 1。此处“优先级”直接由 if / else if 的结构决定。

最重要的一句解释

“count 的下一状态 = 复位时为 0；否则使能时为旧 count 加 1；否则为旧 count。”这就是从代码提炼状态转移规则。

练习：把计数器改成每次加 2，需要改哪里？4 位计数从 14 加 2 后是多少？为什么？

1.6 仿真与测试平台

怎样知道写出的行为符合预期

仿真器在计算机上模拟硬件描述的行为。测试平台 testbench 是专门给模块提供输入并检查输出的代码。被测模块简称 DUT，即 design under test。

配套文件中 counter4.sv 是设计；counter4_tb.sv 是测试平台。测试平台提供时钟，依次驱动 reset 和 enable，等待采样沿，然后比较 count 与预期值。

```verilog
reg clk = 0;
always #5 clk = ~clk;
```

这里 #5 表示仿真延时，~ 表示按位取反。结合测试文件开头的 timescale 1ns/1ps，时钟每 5 纳秒翻转一次，周期为 10 纳秒。此处 # 延时属于测试激励，不用来描述一般可综合的计数器硬件。

为什么在沿后稍等再检查

```verilog
@(negedge clk); reset = r; enable = e;
@(posedge clk); #1;
if (count !== expected) $fatal(1, "counter mismatch");
```

先在下降沿改变输入，让输入在下一上升沿前稳定；再等上升沿并留出仿真更新时间检查。这样避免测试程序与设计在同一仿真时刻争抢读写，产生不容易理解的结果。

!== 还会区分未知值 X 和高阻态 Z；未知的 count 不会被误当成正确。$fatal 表示测试失败并终止；$display 打印信息；$finish 正常结束仿真。

测试至少覆盖什么

- reset 与 enable 同时为 1：确认优先级。
- enable=0：确认保持旧值。
- enable=1：确认加 1。
- count 从 15 加 1：确认位宽引起的回绕。

PASS 表示这些被检查的情形通过。它不等于所有可能行为已经被证明正确，也不包含时序收敛与 FPGA 实测。

1.7 按操作系统准备仿真工具

macOS：终端 + Homebrew

先按 05 安装 Homebrew，再在 Mac 终端执行：

```bash
brew install python icarus-verilog
```

注意包名是 icarus-verilog，装好后命令名为 iverilog 与 vvp。Apple Silicon 与 Intel 使用自己的原生工具架构；不要手工复制 Windows 的可执行文件。[V2]

Windows：PowerShell 或 WSL

如果已有原生 Icarus，确认 `Get-Command iverilog,vvp` 能找到两个工具，即可运行本册代码。没有原生工具可进入 WSL Ubuntu，按下方 Ubuntu 步骤准备。Windows 与 WSL 的命令分别在各自终端输入。

Ubuntu / WSL：Ubuntu 终端

```bash
sudo apt update
sudo apt install python3 iverilog
```

准备好后选自己的入口

先进入“配套示例”目录，按系统任选一条：

```powershell
# Windows PowerShell
py -3 run_examples.py rtl
```

```bash
# macOS Terminal
bash macOS/run_examples.sh rtl
```

```bash
# Ubuntu / WSL Ubuntu
bash Ubuntu-WSL/run_examples.sh rtl
```

三个入口都检查选择器、计数器、手写停止门与已附带的 Chisel 生成模块。仿真输出统一写入 `.outputs/`。完整定位目录和安装步骤见 05。

1.7 运行并理解仿真结果

编译与运行，三个系统使用同样的参数

安装好 Icarus 并进入“配套示例”后，可以手工逐个运行：

```TEXT
iverilog -g2012 -s counter4_tb -o counter4.vvp counter4.sv counter4_tb.sv
vvp counter4.vvp
```

Windows 在 PowerShell、macOS 在终端、Ubuntu 在 Ubuntu 终端运行上面的两行。iverilog 负责编译设计与测试，vvp 执行仿真。[V1]

命令逐项解释

部分	含义
-g2012	按 SystemVerilog 2012 模式处理
-s counter4_tb	指定测试平台为顶层
-o counter4.vvp	指定编译结果文件
后面的两个 .sv	编译需要的设计文件和测试文件

预期能看到计数结果，最后一行为：

```TEXT
PASS counter4: reset/hold/increment/wrap
```

手工命令在当前目录生成 counter4.vcd 波形文件；入口脚本运行则生成在 .outputs/。有兼容的波形查看器即可打开，观察 clk、reset、enable、count。没有查看器也可先读文本输出与逐拍表。

选择器和安全门的运行方式相同，只需替换文件名和顶层名：

```BASH
iverilog -g2012 -s mux2_tb -o mux2.vvp mux2.sv mux2_tb.sv
vvp mux2.vvp
iverilog -g2012 -s safety_gate_tb -o gate.vvp safety_gate.sv safety_gate_tb.sv
vvp gate.vvp
```

常见错误：“找不到模块”时检查文件是否都传给编译器；“找不到命令”时检查安装；“找不到文件”时检查当前目录。先保存源文件，再重新编译，最后重新运行。

1.8 状态机与停止锁存

把“曾经收到停止命令”记下来

有限状态机 FSM 可以理解为“有限个状态，加上一组切换规则”。本例只有 RUN 与 STOP 两个状态，用一个 bit 保存：stopped=0 表示 RUN，stopped=1 表示 STOP。

```text
RUN --收到 kill_req 的采样沿--> STOP
STOP --无论 kill_req 是否撤回--> STOP
STOP --受信任的复位-----------> RUN
```

“锁存停止”在这里是功能描述，具体实现采用上升沿触发的寄存器，并没有使用电平敏感锁存器。

```verilog
always @(posedge clk) begin
  if (reset) stopped <= 1'b0;
  else if (kill_req) stopped <= 1'b1;
end
assign allow = request && authorized && !stopped;
```

一旦采样到 kill_req=1，stopped 变为 1。后续即使 kill_req 回到 0，寄存器也保持 1，因此继续阻止 allow。

采样沿	reset	request	authorized	kill_req	沿后 stopped	沿后 allow
1	1	0	0	0	0	0
2	0	1	1	0	0	1
3	0	1	1	1	1	0
4	0	1	1	0	1	0

注意本例在采样沿之后才进入 STOP。若 kill_req 在两次采样沿之间变化，不能宣称输出在任何物理时刻都立即归零。真正跨时钟域或紧急停止电路还需同步、可靠触发和完整的时序设计。

练习：如果把停止状态直接写成 `assign stopped = kill_req;`，当命令撤回时会怎样？这与当前需求有何区别？

1.9 从一个小门电路看到系统问题

停止标志只是控制链上的一环

安全门示例只控制 allow 这一条信号。它没有操作系统接口、Agent 身份识别、网络接口或存储控制，因此不会实际终止任何 Agent。

把它放入一条假想链路：

```text
受信任的停止命令
        |
        v
停止状态寄存器 --> 拒绝新请求 --> 系统接口执行停止
                                          |
                                          v
                                确认相关任务和副作用已收敛
```

需要继续问的五个问题

1. 谁能写停止位、谁能复位？恶意程序能否把它清掉？
2. allow=0 后，已经进入队列或已经发出的请求怎样处理？
3. CPU 停止后，GPU、DMA、远端服务是否仍会工作？DMA 可理解为设备自行搬运内存数据的能力。
4. 同一个 Agent 的多个进程如何关联到同一个控制域？
5. “停止完成”由谁确认，确认的范围和时间基准是什么？

本例中 reset 优先于 kill_req。两者同时为 1，会把 stopped 清零；若此时请求与授权都为 1，allow 可为 1。测试专门覆盖这个行为，是为了让你看到“优先级”会影响安全含义。生产设计必须明确复位权限与冲突策略。

小改动练习

需求变成“停止命令与复位同时出现时，停止优先”。应把判断顺序改为：

```verilog
if (kill_req) stopped <= 1'b1;
else if (reset) stopped <= 1'b0;
```

这也会改变启动阶段的行为，需要同步修改规范和测试。不要只改代码，不改预期。

> 与课题相关的思考：硬件可以提供更难被篡改的状态和检查点，但“检查点覆盖了哪些行为”仍需要系统层面设计。

1.10 自测与动手题

不看答案，独立完成

1. “把 q 改成 1”与“每个上升沿把 q 更新为 d”有什么不同？
2. 组合电路为什么要说明所有输入分支下的输出？
3. `wire [7:0] x` 有几位？若按无符号数解释，范围是多少？
4. 对本册计数器，依次输入 (reset, enable)：(1,0)、(0,1)、(0,1)、(0,0)、(1,1)，沿后计数分别是多少？
5. 两个寄存器初值 a=0、b=1，在同一个沿执行 `a <= b; b <= a;`，沿后是什么？
6. 停止门中 kill_req 从 1 回到 0，stopped 为什么不回到 0？
7. 停止门何时复位？reset 与 kill_req 同时为 1 时会怎样？
8. 测试输出 PASS，是否代表 FPGA 一定满足时序要求？说明理由。

动手 A：饱和计数器

把 4 位计数器改成达到 15 后保持 15。先写自然语言规则，再补代码：

```verilog
if (reset) count <= 4'd0;
else if (enable && ________) count <= count + 4'd1;
```

测试至少包括：14 加 1 到 15；15 再加仍为 15；禁用时保持；复位时回到 0。

动手 B：自己设计一个状态位

要求“连续运行时允许请求，一旦违规就保持禁用，只有受信任的恢复命令能重新启用”。先定义输入与状态，再写状态转移表。最后明确：违规和恢复同时发生时，哪一个优先？为什么？

本册完成标准

能解释计数器每一行的作用；能不运行程序写出逐拍表；能描述停止门的能力边界；能成功运行示例或保留一份完整的手工结果。

1.11 参考答案与复习卡

前面练习的答案

二进制：0101=5；5 位最大 31；4 位的 15+1 存回后为 0000。限制实例数量通常应避免计数回绕造成低估，可使用饱和计数并单独处理释放与计数一致性。

布尔逻辑：两个 allow 都是 0。“或”规则会使 request=1、authorized=0 也被放行，违反授权要求。反选选择器写为 `sel ? a : b`。

同步复位：若高电平没有覆盖任何采样上升沿，就不保证寄存器看到它。计数每次加 2 可把增量改成 `4'd2`；14+2 截断到 4 位后为 0。

停止状态若直接连 kill_req，则命令撤回后会解除停止，失去“记录已经停止”的能力。

自测答案

1. 前者未说明时序和保持条件；后者定义了采样时刻与寄存器行为。
2. 组合逻辑应仅由当前输入决定；不完整赋值可能引入保持旧值的存储行为。
3. 8 位，0 到 255。
4. 0、1、2、2、0。
5. a=1、b=0，两者按更新前的值计算。
6. 未命中更新分支时，寄存器保持旧值。
7. 在 reset=1 的上升沿复位；本例复位优先，stopped=0。
8. 不代表。功能仿真没有完整验证布线延迟、建立保持时间及 FPGA 实际实现。

饱和计数器空白可填 `count != 4'd15`。动手 B 没有唯一答案，但必须定义优先级、复位值、恢复权限，以及所有状态下的输出。

记住这六句

组合电路看当前输入；时序电路还看保存的状态。位宽决定表示范围。时钟沿决定采样时刻。复位建立已知状态。测试平台主动制造输入并检查输出。一个硬件标志位的正确性，不等于整个系统安全目标已经实现。

阅读来源

[V1] Icarus 入门：https://steveicarus.github.io/iverilog/usage/getting_started.html

[V2] macOS Icarus 安装：https://formulae.brew.sh/formula/icarus-verilog

[C3] Chisel 复位说明：https://www.chisel-lang.org/docs/explanations/reset

[C4] Chisel 时序电路：https://www.chisel-lang.org/docs/explanations/sequential-circuits

本册代码与练习为教学编写，完整测试文件位于“配套示例”。阅读来源用于核对工具和语言概念，没有照搬其完整教程。

语法补充 V1：读懂模块与信号

必学：先分清哪是名字，哪是语法

```VERILOG
// This is a single-line comment.
module gate8(
  input wire [7:0] data,
  input wire enable,
  output wire [7:0] out
);
  assign out = enable ? data : 8'd0;
endmodule
```

这是完整的组合模块。gate8、data、enable、out 都是作者起的名字；module、input、wire、assign 是语言规定的关键字。大小写敏感，enable 与 Enable 是两个名字。

模块端口放在圆括号内，各项用逗号分开，最后一项不加逗号；声明和赋值一般以分号结束。// 后到行末是注释，/* ... */ 是块注释，不参与电路功能。

wire、reg、logic 怎么区分

写法	本周怎样理解	注意什么
wire	连接信号的网络	常接 assign 或子模块输出
reg	传统 Verilog 的过程赋值变量	在 always 中赋值，不必然是寄存器
logic	SystemVerilog 的常用四态数据类型	可描述组合或时序变量，仍需看驱动方式
integer	整数变量	测试中常用于循环下标

本资料的 .sv 文件启用 SystemVerilog，但保留 wire / reg 以便你对照原有经验。初学时遵循“一个信号由一处明确逻辑驱动”，不要让多个 always 随意写同一变量。

位宽与常量速查

[7:0] 是从编号 7 到 0，共 8 位；最高编号不等于位数。8'd12、8'h0C、8'b00001100 表示同一个 8 位无符号值。d、h、b 分别代表十进制、十六进制和二进制。

1'b0 是一个 bit 的 0；8'bxxxx_xxxx 表示仿真中的未知位，z 表示高阻态。X 不是可以随便当作 0 使用的“默认值”。初学示例尽量明确写出位宽和复位。

小练习：把 gate8 改成 4 位需要改哪些地方？答案：两个 [7:0] 改为 [3:0]，零常量改为 4'd0，模块名可自行改为 gate4。

语法补充 V2：运算、取位与拼接

必学：相似的符号，可能算出不同结果

写法	含义	本周常见用途
+、-	加、减	计算下一状态
==、!=	相等、不相等	判断状态
===、!==	含 X / Z 的严格比较	测试输出
<、<=、>、>=	大小比较	上限判断
&&、!	逻辑与、逻辑非	判断条件
&、~、^	按位与、非、异或	逐 bit 计算

逻辑或写 ||，按位或写 |；左移和右移写 <<、>>。逻辑运算判断是否为零，按位运算逐位计算；对多位数据，两者结果可能不同。

```verilog
// Assume a = 4'b1010 and b = 4'b0101.
assign bit_result = a & b;       // 4'b0000
assign logical_result = a && b; // 1'b1
assign low_nibble = data[3:0];
assign highest_bit = data[7];
assign joined = {a, b};         // 8'b10100101
assign repeated = {4{1'b1}};    // 4'b1111
```

花括号表示拼接，左边参数放高位。a、b 各 4 位，{a,b} 共 8 位；{4{1'b1}} 得到四个 1。

三目选择与括号

```verilog
assign out = enable ? data : 8'd0;
assign allow = request && authorized && !stopped;
```

三目表达式描述选择器。复杂条件用括号明确分组，例如 (a == b) && enable。

保留进位时要明确扩宽

若 a、b 各 8 位，本例使用 {1'b0,a} + {1'b0,b} 得到 9 位加法。240+60=300，8 位结果只保留 44，9 位结果可以保存 300。不要仅凭常量写法猜测表达式位宽。

语法补充 V3：赋值与条件分支

必学：先问是否需要保存状态

```VERILOG
reg [7:0] y;
always @(*) begin
  y = 8'd0;
  if (enable) begin
    y = a;
  end
end
```

这是模块内部的组合逻辑片段，假定 enable、a 已声明。@(*) 表示相关输入变化时重新求值；先给默认值，再按条件覆盖。enable=0 时输出也是确定的，不会无意保留旧值。

如果有多种选择，可用 case：

```VERILOG
always @(*) begin
  case (op)
    2'd0: y = a;
    2'd1: y = b;
    default: y = 8'd0;
  endcase
end
```

case ... endcase 按值选择，default 处理未列出的情况。begin ... end 将多条语句组成一个块；if 只控制下一条语句或紧随的整个块。

时序逻辑用时钟沿触发

```VERILOG
always @(posedge clk) begin
  if (reset) q <= 8'd0;
  else if (enable) q <= d;
end
```

此处 <= 是非阻塞赋值：在采样沿根据旧状态计算，再安排状态更新。未命中分支时寄存器保持旧值。对比 if (a <= b) 中的 <=，那里是“小于等于”比较。

入门习惯：组合过程用 =，时钟过程用 <=；不要对同一个变量混用两者。SystemVerilog 常用 always_comb 对应组合过程，always_ff @(posedge clk) 对应时钟过程，变量常声明为 logic。

课题联系：停止与恢复谁优先，由 if / else if 顺序决定。先写清规则，再写分支，不要靠看起来相似的代码猜行为。[V3]

语法补充 V4：实例、参数与循环

必学：模块定义一次，可以使用多次

```VERILOG
mux2 u_select(
  .a(left), .b(right),
  .sel(choose), .y(result)
);
```

这是已有 mux2 的实例化片段。mux2 是模块类型，u_select 是这一个实例的名字。.a(left) 表示“子模块端口 a 接当前模块信号 left”；点左侧是子模块端口名，括号里是本层信号。

配套仿真文件里的 dut 也是实例名，常表示被测设计。SystemVerilog 的 dut(.*) 表示按同名自动连接；初学先用显式端口连接读清楚，再理解缩写。

parameter：构建时决定电路规模

```VERILOG
module pass_data #(parameter WIDTH = 8)(
  input wire [WIDTH-1:0] in_data,
  output wire [WIDTH-1:0] out_data
);
  assign out_data = in_data;
endmodule
```

完整模块默认是 8 位。实例化时可用 pass_data #(.WIDTH(4)) u_pass(...) 改成 4 位，省略号处需补实际端口。参数在构建电路时确定，不是运行中每个时钟沿可随意改变的控制信号。[V4]

选读：generate 的循环生成

```VERILOG
genvar i;
generate
  for (i = 0; i < 4; i = i + 1) begin : g_copy
    assign out_bits[i] = in_bits[i];
  end
endgenerate
```

假定输入输出各 4 位。这是生成四条连接，不是让一根导线在四个时钟内依次工作。仿真文件中的普通 for 循环若带 #1，则可按仿真时间依次制造输入。

不要用“出现了 for”判断是否消耗多个周期；要看它位于生成结构、组合逻辑还是测试过程，以及是否有明确状态与时序控制。

语法补充 V5：读懂你的仿真文件

必学：testbench 就是你以前写的仿真文件

```verilog
`timescale 1ns/1ps
module clock_demo_tb;
  reg clk = 0;
  always #5 clk = ~clk;
  initial begin
    $display("simulation starts");
    repeat (3) @(posedge clk);
    $finish;
  end
endmodule
```

这个完整小例子只演示时钟与等待。反引号开头的 timescale 设置时间单位和精度，不是普通引号。每隔 5 ns 翻转时钟，因此周期 10 ns；repeat 等待三个上升沿，再结束。

你在配套文件中会看到的语法

写法	它在仿真中做什么	怎样读
initial begin ... end	该过程从仿真开始执行一次	初始化与测试步骤
always #5 ...	无限重复翻转时钟	测试时钟发生器
#1	等待一个时间单位	此处为 1 ns
@(posedge clk)	等到时钟上升沿	先等待，再执行后面语句
task ... endtask	定义可重复调用的测试步骤	配套 tick 是测试作者起的名字
$display	打印文本和数值	%0d 十进制，%b 二进制
$fatal、$finish	报错终止、正常结束	便于自动判断结果

$dumpfile 与 $dumpvars 记录 VCD 波形；它们不产生被测电路的功能。# 延时、打印和任务中的等待在这里用于测试，不应直接当成一般可综合硬件的设计方式。

配套 tick 为什么先等下降沿

它在下降沿改变输入，下一上升沿由设计采样，再稍后检查输出。这样避免测试过程与寄存器更新在同一时刻互相争抢。task 是为减少重复测试代码，不意味着产生一块名叫 tick 的硬件。

练习：时钟周期是 10 ns，执行 repeat(2) @(posedge clk) 表示等两个上升沿，还是把时钟周期改成 2 ns？答案：等两个上升沿。

语法补充 V6：综合运用的小模块

运行一次，把语法与波形连接起来

配套 syntax_basics.sv 同时展示组合计算与寄存器更新；下面是其主要逻辑，接口见完整文件：a、b 是 8 位，op 是 2 位，y 是 8 位，sum_full 是 9 位，count 是 4 位。

```VERILOG
assign sum_full = {1'b0, a} + {1'b0, b};
always @(*) begin
  y = 8'd0;
  case (op)
    2'd0: y = a + b;
    2'd1: y = a & b;
    2'd2: y = {a[3:0], b[3:0]};
    default: y = 8'd0;
  endcase
end
always @(posedge clk) begin
  if (reset) count <= 4'd0;
  else if (enable) count <= count + 4'd1;
end
```

设 a=8'hF0（240），b=8'h3C（60），先手算：

op	y 应是多少	原因
0	8'h2C，即 44	300 存到 8 位，只保留低 8 位
1	8'h30，即 48	按位与
2	8'h0C，即 12	a 的低四位放高半字节，b 的低四位放低半字节
3	0	命中 default

sum_full 始终为 300，不受 op 影响。count 只在采样沿更新；组合计算 y 不必等待时钟。

在已配置好的 VS Code 中运行

“终端 → 运行任务 → 语法：Verilog 综合示例”，或在 WSL 的示例目录运行：

```BASH
python3 run_examples.py rtl --example syntax_basics
```

完整仿真文件为 syntax_basics_tb.sv。预期 PASS，输出还说明检查了 262144 组组合输入及计数行为。波形写入 .outputs/syntax_basics.vcd。

语法补充 V7：自测、答案与速查

不看答案，先解释五处语法

1. wire [7:0] data 为什么是 8 位？data[3:0] 又是多少位？
2. a=4'b1010、b=4'b0101，a&b 和 a&&b 有何不同？
3. .enable(run) 中，哪一个名字属于子模块，哪一个属于当前层？
4. 给寄存器写 if (q <= 8'd10) q <= q + 8'd1;，两个 <= 分别是什么？
5. 在组合 always 中只写 if、不写 else 或默认值，可能出现什么？
6. 模块里有一个 for，就一定每个时钟只处理一个元素吗？

参考答案

1. 编号从 7 到 0 共 8 个；低四位取片得到 4 位。
2. 按位与得到 4'b0000；逻辑与因两个操作数均非零而得到 1'b1。
3. enable 是子模块端口，run 是本层信号。
4. 圆括号里是大小比较，赋值位置是非阻塞赋值。
5. 可能让输出保存旧值，推断无意的锁存器；应完整定义组合输出。
6. 不一定。生成循环或可展开的组合逻辑可以形成并行硬件；逐周期处理要明确设计状态和时钟更新。

写代码时的五步检查

先标注端口和位宽；再区分组合输出与寄存器；为组合逻辑定义完整输出；为状态明确复位和更新条件；最后写仿真步骤并核对预期值。

同一个错误不要只改报错行

例如 “unknown module” 要检查模块定义是否加入编译；“port not found” 要核对点左边的端口名；位宽告警要检查丢弃高位是否符合需求。修正后重新保存、编译、运行。

补充来源

[V3] AMD Verilog 语言与过程：https://docs.amd.com/r/en-US/ug901-vivado-synthesis/Verilog-Language-Support

[V4] AMD 参数说明：https://docs.amd.com/r/2025.1-English/ug901-vivado-synthesis/Verilog-Parameters

本补充使用基础 Verilog 与少量 SystemVerilog；运行采用 -g2012 模式。零散片段说明假定已有接口，完整源文件才是直接运行入口。